МФТИ
ФРКТ

Домашняя работа по теме IR

Курс: *Введение в тензорные компиляторы*

Вариант 1

Студент: Остафейчук Роман
Группа: Б01-401

9 марта 2026 г.

Содержание

1 Постановка задачи **2**

2 Исходный код программы **2**

2.1 Код по варианту 2

2.2 Краткое описание программы 2

3 Граф потока управления (CFG) **3**

3.1 Выделение базовых блоков 3

3.2 CFG 4

3.3 Комментарий к CFG 4

4 Дерево доминаторов **5**

4.1 Краткий анализ 6

5 Граф фронта доминирования **6**

5.1 Краткий анализ 7

6 Промежуточное представление Generic IR **8**

6.1 IR без SSA 8

6.2 Generic IR в SSA-форме 9

6.3 Комментарий к SSA 9

7 Оптимизации **10**

7.1 Оптимизация 1: Constant Folding 10

7.2 Оптимизация 2: LICM 12

7.3 Оптимизация 3: Constant Propagation + Algebraic Simplification 13

7.4 Оптимизация 4: LSR (Loop Strength Reduction) 14

8 Заключение **16**

1 Постановка задачи

В данной работе требуется для функции `function1`:

1) кратко описать, что делает программа;
2) построить граф потока управления;
3) построить дерево доминаторов;
4) построить граф фронта доминирования;
5) построить промежуточное представление Generic IR в SSA-форме;
6) предложить и последовательно применить 2–3 оптимизации (я сделал 4), включая хотя бы одну цикловую.

2 Исходный код программы

2.1 Код по варианту

```
int function1(int* a, int n) {
    int result = 0;
    for (int i = 0; i < n; ++i) {
        int x = 5;
        for (int j = 0; j < n; ++j) {
            if (j % 2 == 0)
                continue;
            result += 2 * 3 * a[j] * x - i - 2;
        }
    }
    return result;
}
```

2.2 Краткое описание программы

Функция `function1` принимает указатель на массив целых чисел `a` и размер массива `n`. Внутри функции используется накопительная переменная `result`, которая изначально равна нулю.

Внешний цикл перебирает индекс `i` от `0` до `n - 1`. На каждой итерации внешнего цикла создаётся локальная константная переменная `x = 5`. Затем внутренний цикл перебирает индекс `j` от `0` до `n - 1`. Если `j` чётный, выполняется переход `continue`, и вычисление для этого значения `j` пропускается. Если `j` нечётный, к `result` прибавляется выражение

$$2 \cdot 3 \cdot a[j] \cdot x - i - 2.$$

Таким образом, функция суммирует значения указанного выражения по всем итерациям внешнего цикла и только по нечётным значениям `j` внутреннего цикла, после чего возвращает итоговую сумму `result`.

3 Граф потока управления (CFG)

3.1 Выделение базовых блоков

Ниже приведены базовые блоки, выделенные для функции `function1`.

Блок	**Содержимое**
bb0	Инициализация `result = 0`, `i = 0`, переход к проверке внешнего цикла
bb1	Проверка условия внешнего цикла `i < n`
bb2	Начало тела внешнего цикла: инициализация `x = 5`, `j = 0`
bb3	Проверка условия внутреннего цикла `j < n`
bb4	Проверка условия `j % 2 == 0`
bb5	Ветка `continue`: инкремент `j` и возврат к условию внутреннего цикла
bb6	Основное вычисление, обновление `result`, инкремент `j`, переход к условию внутреннего цикла
bb7	Инкремент `i` и возврат к условию внешнего цикла
bb8	Возврат значения `result`

3.2 CFG

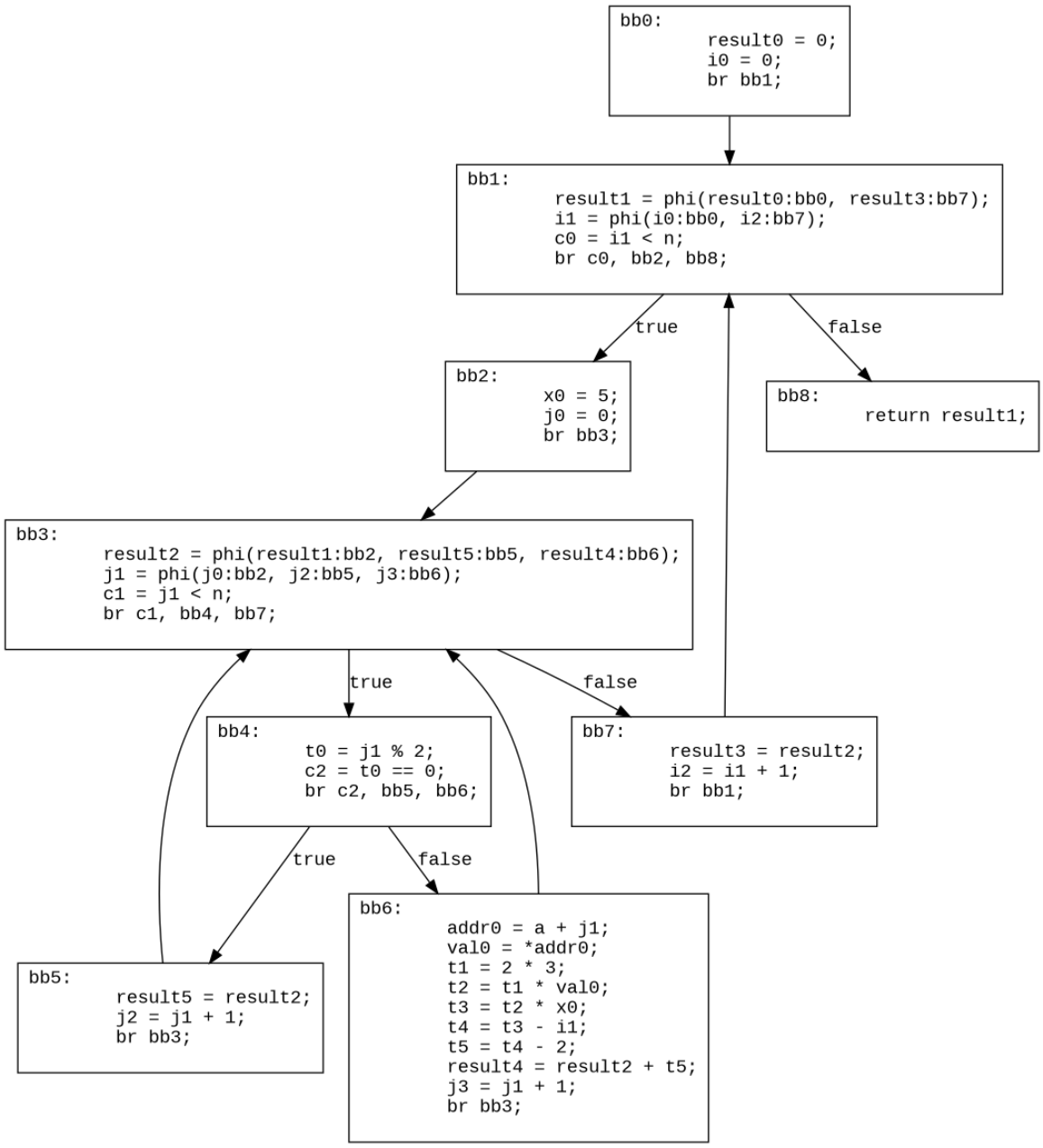

Рис. 1: Граф потока управления функции `function1`

3.3 Комментарий к CFG

Входом в граф является блок `bb0`, в котором выполняются начальные присваивания. Выходом служит блок `bb8`, содержащий инструкцию `return`.

Заголовком внешнего цикла является блок `bb1`, так как именно в нём проверяется условие `i < n`. Заголовком внутреннего цикла является блок `bb3`, в котором проверяется условие `j < n`. Блок `bb4` реализует ветвление по условию чётности индекса `j`.

Таким образом, внешний цикл образуют блоки `bb1`, `bb2`, `bb3`, `bb4`, `bb5`, `bb6`, `bb7`, а внутренний цикл — блоки `bb3`, `bb4`, `bb5`, `bb6`. Обратные дуги `bb7` $\rightarrow$ `bb1` и `bb5` $\rightarrow$ `bb3`, `bb6` $\rightarrow$ `bb3` отражают циклическую структуру программы.

4 Дерево доминаторов

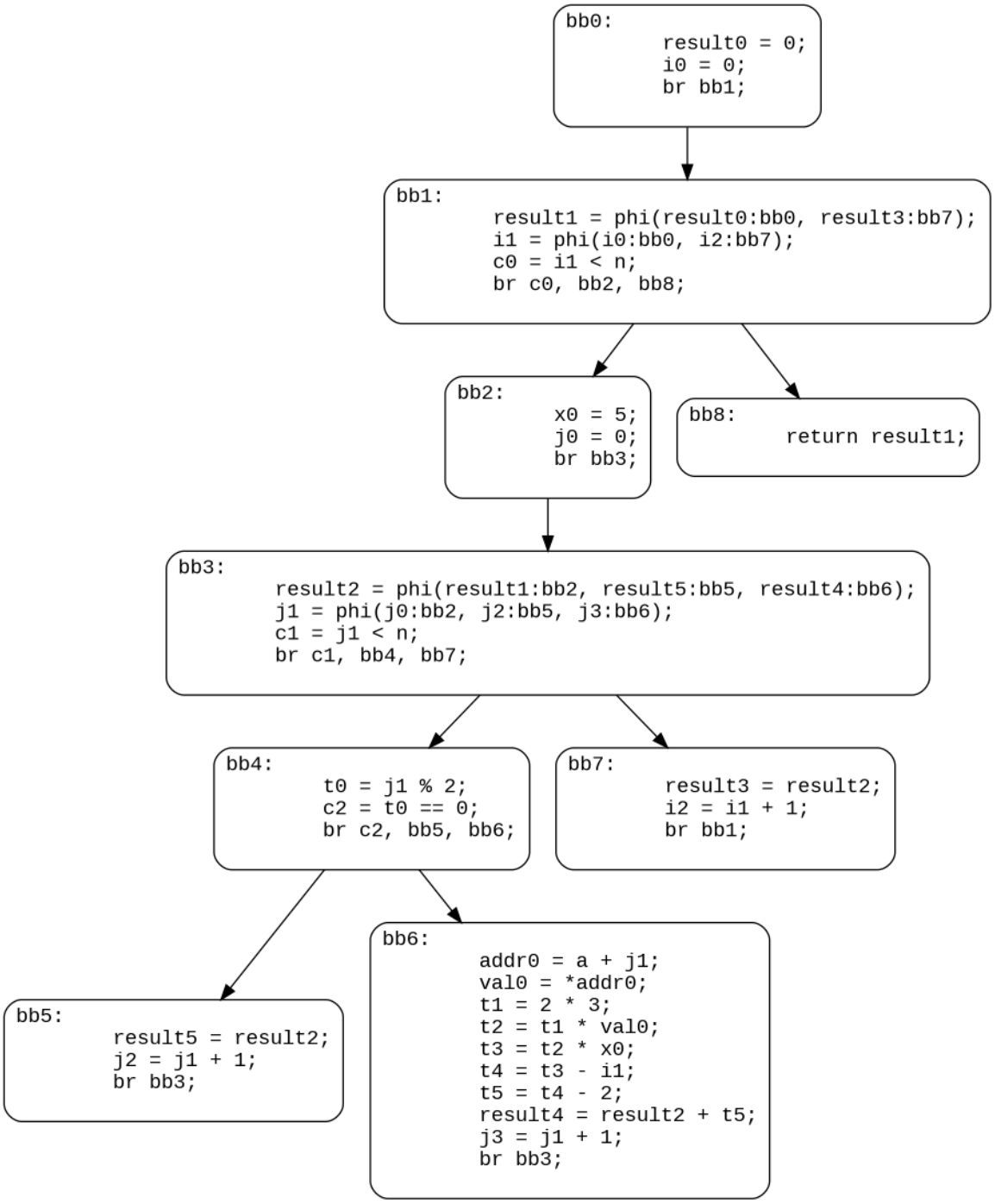

Рис. 2: Дерево доминаторов

4.1 Краткий анализ

Для построенного CFG непосредственные доминаторы имеют следующий вид:

- `idom(bb1) = bb0`;
- `idom(bb2) = bb1`;
- `idom(bb8) = bb1`;
- `idom(bb3) = bb2`;
- `idom(bb4) = bb3`;
- `idom(bb7) = bb3`;
- `idom(bb5) = bb4`;
- `idom(bb6) = bb4`.

Блок `bb1` доминирует над всем телом внешнего цикла, а блок `bb3` доминирует над всем телом внутреннего цикла. Это важно для построения SSA-формы, так как позволяет определить точки, в которых различные версии переменных могут сходиться, и тем самым определить места вставки `phi`-функций.

5 Граф фронта доминирования

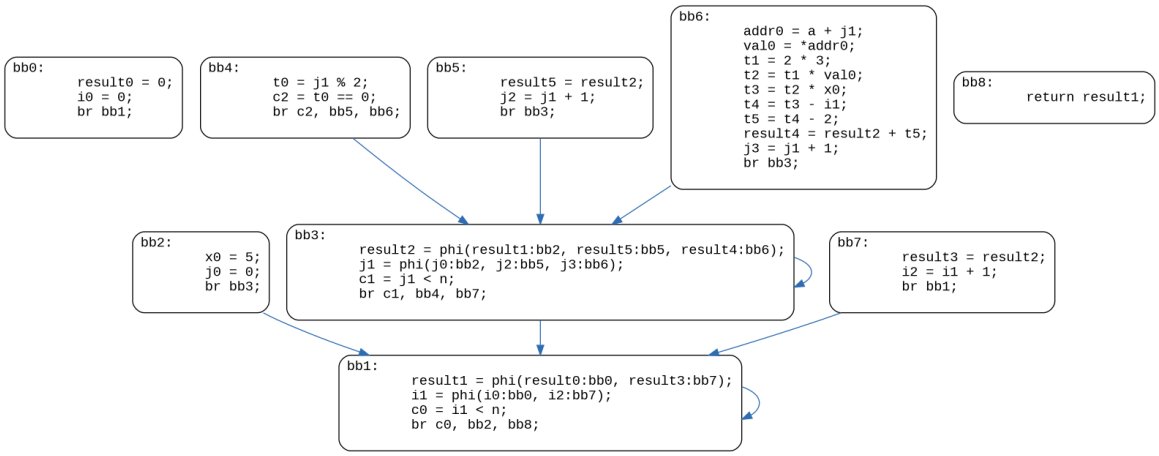

Рис. 3: Граф фронта доминирования

5.1 Краткий анализ

Для рассматриваемого графа существенные множества фронта доминирования имеют вид:

- `DF(bb1) = {bb1}`;
- `DF(bb2) = {bb1}`;
- `DF(bb3) = {bb1, bb3}`;
- `DF(bb4) = {bb3}`;
- `DF(bb5) = {bb3}`;
- `DF(bb6) = {bb3}`;
- `DF(bb7) = {bb1}`.

Схождение управления происходит прежде всего в заголовках циклов `bb1` и `bb3`. Именно поэтому для переменных, меняющихся в циклах, требуются `phi`-функции. Для внешнего цикла это переменные `result` и `i`, а для внутреннего цикла — `result` и `j`.

Фронт доминирования позволяет вставлять `phi`-узлы только в действительно необходимых точках слияния управления, а не во все блоки подряд.

6 Промежуточное представление Generic IR

6.1 IR без SSA

```
function1(a, n):
bb0:
    result = 0;
    i = 0;
    br bb1;

bb1:
    c0 = i < n;
    br c0, bb2, bb8;

bb2:
    x = 5;
    j = 0;
    br bb3;

bb3:
    c1 = j < n;
    br c1, bb4, bb7;

bb4:
    t0 = j % 2;
    c2 = t0 == 0;
    br c2, bb5, bb6;

bb5:
    j = j + 1;
    br bb3;

bb6:
    addr0 = a + j;
    val0 = *addr0;
    t1 = 2 * 3;
    t2 = t1 * val0;
    t3 = t2 * x;
    t4 = t3 - i;
    t5 = t4 - 2;
    result = result + t5;
    j = j + 1;
    br bb3;

bb7:
    i = i + 1;
    br bb1;

bb8:
    return result;
```

6.2 Generic IR в SSA-форме

```
function1(a, n):
bb0:
    result0 = 0;
    i0 = 0;
    br bb1;
bb1:
    result1 = phi(result0:bb0, result3:bb7);
    i1 = phi(i0:bb0, i2:bb7);
    c0 = i1 < n;
    br c0, bb2, bb8;
bb2:
    x0 = 5;
    j0 = 0;
    br bb3;
bb3:
    result2 = phi(result1:bb2, result5:bb5, result4:bb6);
    j1 = phi(j0:bb2, j2:bb5, j3:bb6);
    c1 = j1 < n;
    br c1, bb4, bb7;
bb4:
    t0 = j1 % 2;
    c2 = t0 == 0;
    br c2, bb5, bb6;
bb5:
    result5 = result2;
    j2 = j1 + 1;
    br bb3;
bb6:
    addr0 = a + j1;
    val0 = *addr0;
    t1 = 2 * 3;
    t2 = t1 * val0;
    t3 = t2 * x0;
    t4 = t3 - i1;
    t5 = t4 - 2;
    result4 = result2 + t5;
    j3 = j1 + 1;
    br bb3;
bb7:
    result3 = result2;
    i2 = i1 + 1;
    br bb1;
bb8:
    return result1;
```

6.3 Комментарий к SSA

В SSA-форме каждая переменная определяется ровно один раз, поэтому для обычных переменных пришлось ввести версии с индексами: `result0`, `result1`, `result2`, `i0`, `i1`,

`i2`, `j0`, `j1`, `j2`, `j3` и т.д.

`phi`-функции появились в тех блоках, где сходятся разные варианты одной и той же переменной:

- в блоке `bb1`:
  - `result1 = phi(result0:bb0, result3:bb7)`;
  - `i1 = phi(i0:bb0, i2:bb7)`.

  Здесь происходит слияние значений после входа в внешний цикл и после очередной его итерации;
- в блоке `bb3`:
  - `result2 = phi(result1:bb2, result5:bb5, result4:bb6)`;
  - `j1 = phi(j0:bb2, j2:bb5, j3:bb6)`.

  Здесь происходит слияние значений после входа во внутренний цикл, после ветки `continue` и после ветки с основным вычислением.

Именно блоки `bb1` и `bb3` являются естественными точками схождения управления, что согласуется с построенным графом фронта доминирования.

7 Оптимизации

В этом разделе последовательно применяются четыре оптимизации:

- Constant Folding;
- LICM (Loop Invariant Code Motion);
- Constant Propagation + Algebraic Simplification
- LSR (Loop Strength Reduction).

7.1 Оптимизация 1: Constant Folding

Идея. В блоке `bb6` присутствует константное выражение `2 * 3`. Его значение не зависит от входных данных и может быть вычислено на этапе компиляции.

IR после Constant Folding:

```
function1(a, n):
bb0:
    result0 = 0;
    i0 = 0;
    br bb1;
```

```
bb1:
    result1 = phi(result0:bb0, result3:bb7);
    i1 = phi(i0:bb0, i2:bb7);
    c0 = i1 < n;
    br c0, bb2, bb8;

bb2:
    x0 = 5;
    j0 = 0;
    br bb3;

bb3:
    result2 = phi(result1:bb2, result5:bb5, result4:bb6);
    j1 = phi(j0:bb2, j2:bb5, j3:bb6);
    c1 = j1 < n;
    br c1, bb4, bb7;

bb4:
    t0 = j1 % 2;
    c2 = t0 == 0;
    br c2, bb5, bb6;

bb5:
    result5 = result2;
    j2 = j1 + 1;
    br bb3;

bb6:
    addr0 = a + j1;
    val0 = *addr0;
    t1 = 6;
    t2 = t1 * val0;
    t3 = t2 * x0;
    t4 = t3 - i1;
    t5 = t4 - 2;
    result4 = result2 + t5;
    j3 = j1 + 1;
    br bb3;

bb7:
    result3 = result2;
    i2 = i1 + 1;
    br bb1;

bb8:
    return result1;
```

Комментарий. После данной оптимизации устраняется одно лишнее умножение в теле внутреннего цикла, что уменьшает стоимость вычислений.

7.2 Оптимизация 2: LICM

Идея. Инструкция `x0 = 5` является инвариантом внешнего цикла: она не зависит ни от `i`, ни от `j` и всегда вычисляет одно и то же значение. Поэтому её можно вынести из тела внешнего цикла в его preheader, которым в данной функции является блок `bb0`.

IR после LICM:

```
function1(a, n):
bb0:
    result0 = 0;
    i0 = 0;
    x0 = 5;
    br bb1;

bb1:
    result1 = phi(result0:bb0, result3:bb7);
    i1 = phi(i0:bb0, i2:bb7);
    c0 = i1 < n;
    br c0, bb2, bb8;

bb2:
    j0 = 0;
    br bb3;

bb3:
    result2 = phi(result1:bb2, result5:bb5, result4:bb6);
    j1 = phi(j0:bb2, j2:bb5, j3:bb6);
    c1 = j1 < n;
    br c1, bb4, bb7;

bb4:
    t0 = j1 % 2;
    c2 = t0 == 0;
    br c2, bb5, bb6;

bb5:
    result5 = result2;
    j2 = j1 + 1;
    br bb3;

bb6:
    addr0 = a + j1;
    val0 = *addr0;
    t1 = 6;
    t2 = t1 * val0;
    t3 = t2 * x0;
    t4 = t3 - i1;
    t5 = t4 - 2;
    result4 = result2 + t5;
    j3 = j1 + 1;
    br bb3;
```

```
bb7:
    result3 = result2;
    i2 = i1 + 1;
    br bb1;

bb8:
    return result1;
```

Комментарий. После выноса инварианта значение `x0` больше не вычисляется на каждой итерации внешнего цикла, а вычисляется только один раз перед входом в цикл.

7.3 Оптимизация 3: Constant Propagation + Algebraic Simplification

Идея. После предыдущих оптимизаций известны константы `t1 = 6` и `x0 = 5`. Эти значения можно распространить в блок `bb6` и упростить цепочку умножений:

$$(6 \cdot val0) \cdot 5 = 30 \cdot val0.$$

В результате вместо последовательности промежуточных умножений можно использовать одно более простое выражение.

IR после Constant Propagation + Algebraic Simplification:

```
function1(a, n):
bb0:
    result0 = 0;
    i0 = 0;
    x0 = 5;
    br bb1;

bb1:
    result1 = phi(result0:bb0, result3:bb7);
    i1 = phi(i0:bb0, i2:bb7);
    c0 = i1 < n;
    br c0, bb2, bb8;

bb2:
    j0 = 0;
    br bb3;

bb3:
    result2 = phi(result1:bb2, result5:bb5, result4:bb6);
    j1 = phi(j0:bb2, j2:bb5, j3:bb6);
    c1 = j1 < n;
    br c1, bb4, bb7;

bb4:
    t0 = j1 % 2;
    c2 = t0 == 0;
    br c2, bb5, bb6;
```

```
bb5:
    result5 = result2;
    j2 = j1 + 1;
    br bb3;

bb6:
    addr0 = a + j1;
    val0 = *addr0;
    t2 = 30 * val0;
    t3 = t2 - i1;
    t4 = t3 - 2;
    result4 = result2 + t4;
    j3 = j1 + 1;
    br bb3;

bb7:
    result3 = result2;
    i2 = i1 + 1;
    br bb1;

bb8:
    return result1;
```

Комментарий. После распространения констант и алгебраического упрощения выражение в теле внутреннего цикла стало короче и содержит меньше промежуточных вычислений. Это уменьшает число арифметических операций внутри цикла и делает IR более компактным.

7.4 Оптимизация 4: LSR (Loop Strength Reduction)

Идея. Во внутреннем цикле на каждой итерации заново вычисляется адрес элемента массива: `addr0 = a + j1`. Переменная `j1` является индуктивной переменной цикла, так как на каждой итерации увеличивается на единицу. Поэтому вместо повторного вычисления адреса по индексу можно ввести отдельную адресную индуктивную переменную-указатель. Это и есть применение LSR: индексная адресация заменяется на более простое обновление указателя.

IR после LSR:

```
function1(a, n):
bb0:
    result0 = 0;
    i0 = 0;
    x0 = 5;
    br bb1;

bb1:
    result1 = phi(result0:bb0, result3:bb7);
    i1 = phi(i0:bb0, i2:bb7);
    c0 = i1 < n;
    br c0, bb2, bb8;
```

```
bb2:
    j0 = 0;
    p0 = a;
    br bb3;

bb3:
    result2 = phi(result1:bb2, result5:bb5, result4:bb6);
    j1 = phi(j0:bb2, j2:bb5, j3:bb6);
    p1 = phi(p0:bb2, p2:bb5, p3:bb6);
    c1 = j1 < n;
    br c1, bb4, bb7;

bb4:
    t0 = j1 % 2;
    c2 = t0 == 0;
    br c2, bb5, bb6;

bb5:
    result5 = result2;
    j2 = j1 + 1;
    p2 = p1 + 1;
    br bb3;

bb6:
    val0 = *p1;
    t2 = 30 * val0;
    t3 = t2 - i1;
    t4 = t3 - 2;
    result4 = result2 + t4;
    j3 = j1 + 1;
    p3 = p1 + 1;
    br bb3;

bb7:
    result3 = result2;
    i2 = i1 + 1;
    br bb1;

bb8:
    return result1;
```

Комментарий. После применения LSR вычисление адреса элемента массива было заменено на использование отдельной адресной индуктивной переменной `p1`. Благодаря этому внутри цикла больше не требуется каждый раз строить адрес как сумму базы массива и индекса. Такое преобразование уменьшает число операций в цикле и делает промежуточное представление ближе к низкоуровневой реализации.

8 Заключение

В работе была проанализирована функция `function1`, вычисляющая сумму выражения по вложенным циклам с пропуском чётных значений индекса внутреннего цикла. Для данной функции были построены граф потока управления, дерево доминаторов и граф фронта доминирования. Затем было получено промежуточное представление Generic IR в SSA-форме.

После этого были последовательно применены четыре оптимизации: свёртка констант, вынос инварианта цикла, распространение констант с алгебраическим упрощением и LSR. В результате итоговое промежуточное представление стало компактнее, а число вычислений внутри циклов уменьшилось. В частности, вычисление адреса элемента массива во внутреннем цикле было заменено на использование адресной индуктивной переменной.